UNIVERSITÀ DEGLI STUDI DI UDINE
hic sunt futura

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

PROGETTO DI LINGUAGGI E COMPILATORI

Parte 2 - Parziale

Francesco Bortuzzo **157430**
Leonardo Danelutti **158786**
Lorenzo Della Giustina **158854**

Anno Accademico 2024/2025

Indice

1. Definizione del linguaggio: scelte di design e risoluzione dei conflitti 1
1.a. Tipi 1
1.b. Variabili 1
1.c. Costanti 1
1.d. Loop 1
1.e. Condizionali 2
1.f. Routine 2
1.g. Commenti 2
1.h. Separatori 2
1.i. Risoluzione conflitti rimanenti 3
2. Definizione del lexer e del parser 3
2.a. Modifiche al lexer 3
2.b. Modifiche al Parser 3
3. Definizione del Type system 4
3.a. Gerargia dei tipi 4
3.b. Regole formali per il type system 4
3.b.I. Espressioni 5
3.b.II. Statements 7
3.b.III. Dichiarazioni 8
4. Analisi Semantica Statica 9
4.a. Type-checker ed environment 9
4.b. Errori 10
4.b.I. Propagazione degli errori 10
4.b.II. Routine parziali 10
4.b.III. Dangling references 11
4.b.IV. Presenza del main 11
4.c. Casting 11
4.d. Costanti 11
5. Generazione del Three Address Code (TAC) 11
5.a. Estensioni del linguaggio TAC 12
5.b. Design e convenzioni del TAC 12
5.c. Passaggio dei parametri 12
5.d. Dettagli implementativi 13
5.e. Pretty printer 13
Bibliografia 14

1. Definizione del linguaggio: scelte di design e risoluzione dei conflitti

Il linguaggio definito per questo progetto è un linguaggio di programmazione imperativo con scoping statico. La sua grammatica si basa sulla sintassi concreta del linguaggio Blue.

Poiché Blue è un linguaggio orientato agli oggetti, per garantire le funzionalità richieste dalla consegna, come per esempio il dereferenziamento, sono stati introdotti alcuni elementi. Tuttavia, non sono presenti né classi né interfacce.

1.a. Tipi

Il linguaggio definito prevede i seguenti tipi primitivi: `Boolean`, `Integer`, `Real`, `String`[1] e `Char`. Sono inoltre supportati gli array che sono dichiarati utilizzando nel modo seguente: `Array<Type, Integer>`, dove `Type` rappresenta il tipo degli elementi e `Integer` indica la dimensione dell'array. Infine vi sono i puntatori, che sono dichiarati con la sintassi `&Type`.

1.b. Variabili

Le variabili sono dichiarate con la keyword `var` e possono essere di tipo primitivo o di tipo array. Non è consentito dichiarare variabili senza assegnamento[2].

1.c. Costanti

Le costanti sono dichiarate con la keyword `const` e possono essere di tipo primitivo (non devono allocare spazio in memoria). Anche in questo caso non è consentito dichiarare costanti senza assegnamento[3].

1.d. Loop

Per il nostro linguaggio sono previste le forme di iterazione non determinata `while-do` e `do-while`. Entrambe le strutture sono naturalmente implementate dall'elemento `loop-exit on` della sintassi concreta di Blue.

[1] [1] M. Comini, «Front-end di un compilatore per un linguaggio imperativo: note generali», 2024., p. 3.3
[2] [1], p. 3.2
[3] [1], p. 3.2

1.e. Condizionali

Il linguaggio immplementa le strutture condizionali semplici `if-then` e `if-then-else`, che permettono di eseguire blocchi di codice in base a condizioni specifiche.

1.f. Routine

Le routine comprendono sia funzioni che procedure. Una funzione senza parametri può essere invocata sia con la sintassi completa `function()` sia in forma abbreviata `function`, che è semplicemente zucchero sintattico per la prima. Rispetto alla sintassi concreta del linguaggio Blue, sono rimosse pre e post-condizioni; inoltre i parametri di ritorno non vengono nominati esplicitamente nella dichiarazione: si specificano solamente i tipi restituiti mediante la notazione `-> Type`. Si nota che non è possibile restituire tuple. Per gestire il ritorno esplicito dei valori, è stato introdotto il costrutto `return`. Le dichiarazioni di funzioni, in ogni ambiente, devono essere precedute dalla keyword `routines`, in modo simile a quanto avviene in Blue per le dichiarazioni delle routine nelle classi. Questo è necessario per evitare ambiguità, come nel seguente esempio:

```
Blue
var
  a: Integer := b
main ...
```

dove `main` potrebbe rappresentare sia il nome di una funzione che il nome di una variabile.

Quando si invoca una funzione è possibile specificare la modalità di passaggio dei parametri in ingresso (`val` , `valres` e `ref`). Infine, poiché il linguaggio non prevede il concetto di classi, le funzioni non possono essere invocate con la notazione a punto, come `.function()`.

1.g. Commenti

Il linguaggio supporta due tipi di commenti:

- Commenti a riga singola, introdotti dal simbolo `--`, e
- Commenti a blocco, delimitati da `/*` e `*/`.

1.h. Separatori

Al fine di disambiguare la grammatica e evitare conflitti è fondamentale l'introduzione di separatori alla fine degli Statement, salvo If e Loop, e delle dichiarazioni di costanti e variabili. Senza i separatori, infatti, il parser LALR(1) non sarebbe in grado di distinguere con certezza la struttura del codice in determinate situazioni.

Ad esempio:

```
var
  a: Integer := b
( ...
```

In questo caso, con un lookahead di 1 token, non è possibile determinare se `b (` rappresenti l'inizio di una chiamata a funzione oppure se `(` sia l'inizio di una Left Expression.

Un'altra ambiguità possibile è la seguente:

```
var
  a: Integer := b
* ...
```

dove `*` potrebbe essere sia un deref che una moltiplicazione. Per lo stesso motivo è stato aggiunto un `;` anche nell'exit dei loop.

1.i. Risoluzione conflitti rimanenti

La grammatica definita presenta un solo conflitto, che si verifica nello stato 104, tra le produzioni di `LExp` e `RExp`. Questo conflitto non crea problemi per l'analisi sintattica, dal momento che le `LExp` sono un sottoinsieme delle `RExp`. Pertanto, è sufficiente scegliere sempre l'azione di shift.

2. Definizione del lexer e del parser

Lexer e Parser sono stati realizzati modificando i file-prototipo generati automaticamente da BNFC, in modo tale da adattarli alle specifiche della consegna. La grammatica utilizzata per la costruzione di lexer e parser è quella presente nel file `grammar.cf` e genera il linguaggio descritto in precedenza.

2.a. Modifiche al lexer

- Oltre la posizione iniziale di un token è stata aggiunta anche la sua posizione finale, in modo da poter generare errori più precisi.

2.b. Modifiche al Parser

Il parser è stato modificato in modo da utilizzare la sintassi astratta da noi definita nel file `AbsSyn.hs` (a differenza di quella generata da BNFC). La struttura delle regole non è stata alterata.

3. Definizione del Type system

3.a. Gerargia dei tipi

Per il nostro linguaggio, abbiamo definito la seguente gerarchia dei tipi:

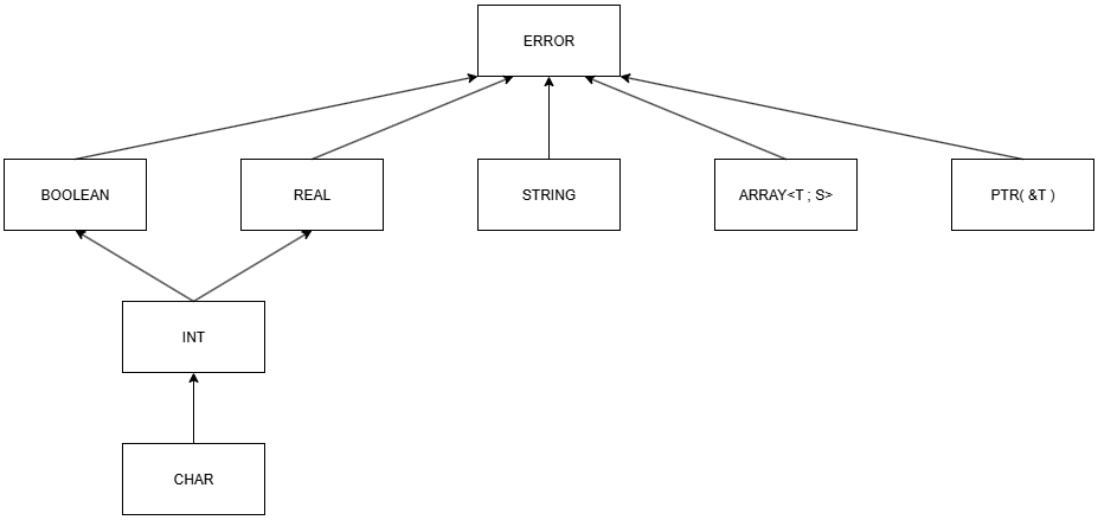


Figura 1: Gerarchia dei tipi.

3.b. Regole formali per il type system

Per definire il tipo di ogni nodo dell'albero di sintassi astratta, di seguito sono riportate le regole formali del type system.

Alcune note:

- L'environemnt, oltre al suo normale scopo, viene usato per propagare la necessità che un'espressione sia di tipo left-hand side. Nelle regole viene omesso, ma tutte le espressioni non-left-hand side tornano un errore se nel loro environment è presente la keyword `LExp`.
- Come spiegato nel paragrafo Sezione 4.a, nell'environemnt i namespace di simboli e routine sono separati. Per semplicità di scrittura nelle regole del type-system il namespace è considerato come unico. Le eventuali ambiguità generano errori o sono gestite usando il contesto (in alcuni casi sono permesse solo left-hand side expression, e quindi non le chiamate a routine).
- Ci sono due modalità con cui vengono mergiati due envirnoment a e b:
  - $a \cup b$: i binding di b sovrascrivono quelli di a;

– $a \uplus b$: in caso di clash viene generato un errore.

3.B.I. Espressioni

Di seguito le regole di annotazione delle espressioni. Si noti che i nodi interni propagano sempre gli errori senza greare «tipi composti da errori» per evitare errori annidati indotti da errori precedenti. Per semplicità di notazione tale processo è omesso nelle seguenti regole.

`LITERAL`

I literal sono foglie dell'AST. Il loro tipo dipende solo dal tipo di nodo.

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitInt } \{\text{intVal} = e\}\} : \text{INT}} \tag{1}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitChar } \{\text{charVal} = e\}\} : \text{CHAR}} \tag{2}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitString } \{\text{stringVal} = e\}\} : \text{STRING}} \tag{3}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitBool } \{\text{boolVal} = e\}\} : \text{BOOL}} \tag{4}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitDouble } \{\text{doubleVal} = e\}\} : \text{REAL}} \tag{5}$$

`IDENT`

Anche gli ident sono foglie dell'AST. Possono essere:

- variabili;
- costanti;
- chiamate a funzioni senza parametri.

In tutti i casi assumono il tipo associato al loro nome nell'ambiente.

$$\frac{}{\text{env} \vdash_E \text{ExpIdent } \{\text{name} = \text{id}\} : \text{env}(\text{id})} \tag{6}$$

`FUNCALL`

Le chiamate di funzione prendono il tipo dall'environment. Gli argomenti supportano il casting implicito solo se passati per valore. Inoltre se usano un metodo di passaggio diverso da quello per valore devono essere espressioni left-hand side.

$$\frac{\text{env}(\text{id}) : \text{mode}_1\tau_1 \times ... \times \text{mode}_n\tau_n \to \tau \qquad \forall i = 1, ..., n \ \text{checkArg}(\text{env}, e_i, \tau_i, \text{mode}_i)}{\text{env} \vdash_E \text{FunCall } \{\text{routCallName} = \text{id}, \text{args} = [e_1, ..., e_n]\} : \text{env}(\text{id})} \tag{7}$$

dove:

$$\begin{gathered}\text{checkArg}(\text{env}, e, \tau, \text{mode}) \\ = \\ \text{env'} \vdash_E e : \tau' \\ \wedge \\ \text{mode} = \text{val} \to (\sup(\tau', \tau) = \tau \wedge \text{env'} = \text{env}) \\ \wedge \\ \text{mode} \neq \text{val} \to (\tau' = \tau \wedge \text{env'} = \text{env} \cup \text{"LExp"})\end{gathered} \tag{8}$$

`SIGN`

Il segno prevede un'espressione numerica: un Integer o un Real. Siccome `sup(Integer, Real) = Real` (come definito in Figura 1), si prova prima a convertire `e` ad `Integer`:

$$\frac{\text{env} \vdash_E \text{exp} : \tau' \qquad \sup(\tau', \text{Integer}) = \text{Integer}}{\text{env} \vdash_E \text{Sign } \{e = \text{exp}\} : \text{Integer}} \tag{9}$$

Se `sup(tau', Integer) != Integer` si prova a convertire `e` a `Real`:

$$\frac{\text{env} \vdash_E \text{exp} : \tau' \qquad \sup(\tau', \text{Real}) = \text{Real} \wedge \sup(\tau', \text{Integer}) \neq \text{Integer}}{\text{env} \vdash_E \text{Sign } \{e = \text{exp}\} : \text{Real}} \tag{10}$$

BINOP

Le operazioni binarie supportate da `blue` si dividono in 4 categorie:

- le operazioni booleane:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Boolean}) = \text{Boolean})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{AND}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Boolean}} \tag{11}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Boolean}) = \text{Boolean})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{OR}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Boolean}} \tag{12}$$

- le operazioni aritmetiche intere:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Integer}) = \text{Integer})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{DIVINT}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Integer}} \tag{13}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Integer}) = \text{Integer})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{MOD}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Integer}} \tag{14}$$

- le operazioni aritmetiche reali:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{DIV}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Real}} \tag{15}$$

- le operazioni aritmetiche intere e reali:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{ADD}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{16}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{SUB}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{17}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{MUL}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{18}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{POW}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{19}$$

BOOLNEG

La negazione prevede un'espressione booleana:

$$\frac{\text{env} \vdash_E \text{exp} : \tau \qquad \sup(\tau, \text{Boolean})}{\text{env} \vdash_E \text{BoolNeg } \{e = \text{exp}\} : \text{Boolean}} \tag{20}$$

RELOP

Le operazioni relazioni supportate sono quelle di uguaglianza e di confronto. Sono supportate da tutti i tipi, purchè convertibili allo stesso supertipo.

$$\frac{\text{env} \vdash_E e_1 : \tau_1 \qquad \text{env} \vdash_E e_2 : \tau_2 \qquad \sup(\tau_1, \tau_2) = \tau}{\text{env} \vdash_E \text{RelOp } \{\text{e1} = e_1, \text{e2} = e_2\} : \text{Boolean}} \tag{21}$$

REF

È consentito creare reference soltanto di espressioni di tipo left-hand side.

$$\frac{\text{env'} \vdash_E \text{exp} : \tau \qquad \text{env'} = \text{env} \cup \text{"LExp"}}{\text{env} \vdash_E \text{Ref } \{e = \text{exp}\} : \&\tau} \tag{22}$$

DEREF

Nei nodi dereference si controlla che l'espressione dereferenziata sia di tipo $\&\tau$.

$$\frac{\text{env} \vdash_{E} \text{exp} : \&\tau}{\text{env} \vdash_{E} \text{Deref } \{e = \text{exp}\} : \tau} \tag{23}$$

ARRAYINDEX

Nell'accesso agli array l'indice è vincolato ad essere un tipo convertibile ad intero, e l'array dev'essere un'espressione di tipo left-hand side.

$$\frac{\text{env'} \vdash_{E} \text{exp} : \text{Array} < \tau, n > \wedge\, \text{env'} = \text{env} \cup \text{"LExp"} \qquad \text{env} \vdash_{E} \text{idx} : \tau' \wedge \sup(\tau', \text{Integer}) = \text{Integer}}{\text{env} \vdash_{E} \text{ArrayIndex } \{e = \text{exp}, \text{index} = \text{idx}\} : \tau} \tag{24}$$

ARRAY

Per creare un nodo array tutti gli elementi dell'array devono avere tipi compatibili tra loro. Il tipo finale contine la grandezza dell'array ed il least common ancestor tra tutti gli elementi.

$$\frac{\forall i = 1, ..., n (\text{env} \vdash_{E} e_i : \tau_i') \wedge \tau_1 = \tau_1' \wedge \forall i = 2, ..., n (\tau_i = \sup(\tau_{i-1}, \tau_i'))}{\text{env} \vdash_{E} \text{Array } \{\text{elements} = [e_1, ..., e_n]\} : \text{Array} < \tau_n, n >} \tag{25}$$

3.B.II. Statements

In `blue` gli statements sono separati dalle dichiarazioni. Quindi non introducono nuovi binding.

STMTASS

Negli assegnamenti viene controllato che:

- l'espressione a sinistra sia un'espressione left-hand side;
- l'espressione a destra possa essere convertita al tipo dell'espressione a sinistra.

$$\frac{\text{env} \vdash_{S} l : \tau_l \wedge \text{isLExp}(l) \wedge \text{isNotConst}(l) \qquad \text{env} \vdash_{S} r : \tau_r \qquad \sup(\tau_l, \tau_r) == \tau_l}{\text{env} \vdash_{S} \text{StmtAss } \{\text{left} = l, \text{right} = r\} : \emptyset} \tag{26}$$

STMTROUTCALL

Gli statement chiamate di routine vengono annotati come le espressioni chiamate di funzione. Il tipo di ritorno è inserito tra parentesi per indicare la compatibilità della regola sia per chiamate a procedure che funzioni.

$$\frac{\text{env(id)} : \text{mode}_1\tau_1 \times ... \times \text{mode}_n\tau_n(\rightarrow \tau) \qquad \forall i = 1, ..., n \text{ checkArg}(\text{env}, e_i, \tau_i, \text{mode}_i)}{\text{env} \vdash_{S} \text{StmtRoutCall } \{\text{call} = \text{RoutineCall } \{\text{routCallName} = \text{id}, \text{args} = [e_1, ..., e_n]\}\} : \emptyset} \tag{27}$$

STMTRETRUN, **STMTRETRUNEMPTY** e **STMTCONTINUE**

Per la keyword `return` si controlla il tipo (se presente) che ha nell'environment.

$$\frac{\text{env} \vdash_{E} e : \tau \wedge \sup(\tau, \text{env(return)}) == \text{env(return)}}{\text{env} \vdash_{S} \text{StmtReturn } \{\text{valReturn} = e\} : \emptyset} \tag{28}$$

$$\frac{\text{env(return)} : \emptyset}{\text{env} \vdash_{S} \text{StmtReturnEmpty } \{\} : \emptyset} \tag{29}$$

Per la keyword `continue` si controlla che sia presente nell'environment.

$$\frac{\text{env(continue)} : \emptyset}{\text{env} \vdash_{S} \text{annStmtContinue } \{\} : \emptyset} \tag{30}$$

STMTIF e **STMTIFELSE**

Negli statement `if _ then _` e `if _ then _ else _` si controlla che la condizione sia compatibile con i booleani.

$$\frac{\text{env} \vdash_E e : \tau \wedge \sup(\tau, \text{Boolean}) == \text{Boolean} \qquad \text{env} \vdash_S s_1, ..., s_n}{\text{env} \vdash_S \text{StmtIf } \{\text{cond} = e, \text{body} = [s_1, ..., s_n]\} : \emptyset} \tag{31}$$

$$\frac{\text{env} \vdash_E e : \tau \wedge \sup(\tau, \text{Boolean}) == \text{Boolean} \qquad \text{env} \vdash_S s_{\text{t1}}, ..., s_{\text{tn}} \qquad \text{env} \vdash_S s_{\text{f1}}, ..., s_{\text{fn}}}{\text{env} \vdash_S \text{StmtIfElse } \{\text{cond} = e, \text{bodyT} = [s_{\text{t1}}, ..., s_{\text{tn}}] \text{ bodyT} = [s_{\text{f1}}, ..., s_{\text{fn}}]\} : \emptyset} \tag{32}$$

`STMTLOOP` and **`STMTLOOPEXIT`**

I loop chiamano ricorsivamente l'annotazione del body e delle condizioni di uscita. Viene aggiunta all'environment la keyword «continue» per segnalare al modulo di analisi semantica statica la possibilità di usare tale keyword.

$$\frac{\text{env} \cup \text{"continue"} \vdash_S s_1, ..., s_{n_s} \qquad \text{env} \cup \text{"continue"} \vdash_S e_1, ..., e_{n_e}}{\text{env} \vdash_S \text{StmtLoop } \left\{\text{body} = \left[s_1, ..., s_{n_s}\right], \text{exit} = \left[e_1, ..., e_{n_e}\right]\right\} : \emptyset} \tag{33}$$

Nelle condizioni di uscita dai loop si controlla che la condizione sia compatibile con i booleani.

$$\frac{\text{env} \vdash_E e : \tau \wedge \sup(\tau, \text{Boolean}) == \text{Boolean} \qquad \text{env} \vdash_S s_1, ..., s_n}{\text{env} \vdash_S \text{LoopExit } \{\text{cons} = e, \text{body} = [s_1, ..., s_n]\} : \emptyset} \tag{34}$$

3.B.III. Dichiarazioni

`PROGRAMDECL`

Un blocco `ProgramDecl` introduce nuove dichiarazioni di costanti, variabili e routine. I binding delle routine sono visibili in tutto il blocco, e segnalano errore in presenza di conflitti ($\uplus$). La dichiarazione di costanti e variabile procede in ordine mascherando eventuali conflitti ($\cup$). L'ordine di merge tra simboli (costanti e variabili) e routine non è importante in quanto namespace separati.

$$\frac{\begin{gathered}\forall i = 1, ..., n_c \Big((\text{env} \uplus \text{env}_r) \cup \cup_0^{i-1} \text{env}_{c_i} \vdash_D c_i : \text{env}_{c_i}\Big) \\ \forall i = 1, ..., n_v \Big(((\text{env} \uplus \text{env}_r) \cup \text{env}_c) \cup \cup_0^{i-1} \text{env}_{v_i} \vdash_D v_i : \text{env}_{c_i}\Big) \\ \forall i = 1, ..., n_r \Big(((\text{env} \uplus \text{env}_r) \cup \text{env}_c) \cup \text{env}_v \vdash_D r_i : \text{env}_{c_i}\Big)\end{gathered}}{\begin{gathered}\text{env} \vdash_D \text{ProgramDecl } \Big\{\text{consts} = \Big[c_1, ..., c_{n_c}\Big], \text{vars} = \Big[v_1, ..., v_{n_v}\Big], \\ \text{routines} = \Big[r_1, ..., r_{n_r}\Big]\Big\} : \text{env}_c \cup \text{env}_v \cup \text{env}_r\end{gathered}} \tag{35}$$

dove:

- env_{c_0}, env_{v_0} e env_{r_0} sono envirnoment vuoti (senza binding);
- $\text{env}_c = \cup_1^n \text{env}_{c_i}$, $\text{env}_v = \cup_1^n \text{env}_{v_i}$ e $\text{env}_r = \cup_1^n \text{env}_{r_i}$.

`CONSTASS`

Nelle dichiarazioni di costanti si possono dichiarare più costanti assieme. L'environment non viene propagato perchè è come se venissero dichiarate «contemporaneamente».

$$\frac{\forall i = 1, ..., n (\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau, \tau_i) = \tau)}{\begin{gathered}\text{env} \vdash_D \text{ConstAss } \{\text{constIdents} = [\text{id}_1, ..., \text{id}_n], \text{constType} = \tau, \\ \text{constValues} = [e_1, ..., e_n]\} : \{\text{id}_1 : \tau_1, ..., \text{id}_n : \tau_n\}\end{gathered}} \tag{36}$$

`VARASS`

Nelle dichiarazioni di variabili si possono dichiarare più variabili assieme. L'environment non viene propagato perchè è come se venissero dichiarate «contemporaneamente».

$$\frac{\forall i = 1, ..., n(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau, \tau_i) = \tau)}{\begin{array}{c}\text{env} \vdash_D \text{ConstAss } \{\text{varIdents} = [\text{id}_1, ..., \text{id}_n], \text{varType} = \tau, \text{varValues} = [e_1, ..., e_n]\} \\ : \{\text{id}_1 : \tau_1, ..., \text{id}_n : \tau_n\}\end{array}} \tag{37}$$

`FUNCTION` e `PROCEDURE` e `ROUTBODY`

Le funzioni e le procedure vengono annotate all stesso modo. In entrambi i casi viene aggiunta all'environment la keyword «return» per segnalare al modulo di analisi semantica statica la possibilità di usare tale keyword (nel caso delle funzioni alla keyword è associato il tipo di ritorno).

$$\frac{\text{env}' \vdash_D b \qquad \text{env}' = \text{env} \cup \{\text{id}_1 : \tau_1, ..., \text{id}_n : \tau_n\} \cup \{\text{"return"} : \tau\}}{\begin{array}{c}\text{env} \vdash_D \text{Function } \{\text{routName} = \text{id}, \text{params} = [\text{id}_1 : \text{mode}_1\tau_1, ..., \text{id}_n : \text{mode}_n\tau_n], \\ \text{returnType} = \tau, \text{body} = b\} : \{\text{id} : \text{mode}_1\tau_1 \times ... \times \text{mode}_n\tau_n \to \tau\}\end{array}} \tag{38}$$

$$\frac{\text{env}' \vdash_D b \qquad \text{env}' = \text{env} \cup \{\text{id}_1 : \tau_1, ..., \text{id}_n : \tau_n\} \cup \{\text{"return"}\}}{\begin{array}{c}\text{env} \vdash_D \text{Procedure } \{\text{routName} = \text{id}, \text{params} = [\text{id}_1 : \text{mode}_1\tau_1, ..., \text{id}_n : \text{mode}_n\tau_n], \\ \text{body} = b\} : \{\text{id} : \text{mode}_1\tau_1 \times ... \times \text{mode}_n\tau_n\}\end{array}} \tag{39}$$

Il boby delle dichiarazioni di routine

$$\frac{\text{env} \vdash_D d : \text{env}' \qquad \text{env}' \vdash_S s}{\text{env} \vdash_S \text{RoutBody } \{\text{declarations} = d, \text{statements} = [s_1, ..., s_n], \} : \emptyset} \tag{40}$$

4. Analisi Semantica Statica

L'analisi semantica statica è implementata nel modulo `StatSem`. Più nello specifico è diviso in 4 sotto-moduli: `Exp`, `Decl` e `Stmt` rispettivamente responsabili dell'annotazione delle espressioni, delle dichiarazioni e degli statement, e `Utils` in cui sono definite utility e tipi di dato usati in tutto il modulo, tra cui le annotazioni, l'environment ed il casting.

4.a. Type-checker ed environment

L'annotazione dei tipi delle espressioni segue le regole del type system definito sopra.

Per poter annotare correttamente i tipi viene propagato un environment a partire dal nodo radice alle foglie (al contrario rispetto a come procede l'annotazione). Il namespace tra routine e simboli (variabili e costanti) è separato. Questo talvolta causa possibili problemi di ambiguità perchè le routine senza parametri possono essere chiamate anche senza `()` (scrivere `main` è zucchero sintattico per `main()`) come nella sintassi concreta di `blue`.

Inoltre è stato aggiunto il namespace `keyword` in cui, oltre a segnalare la possibilità di usare le keyword `return` e `continue`, si può richiedere che l'espressione sia di tipo left-hand side. Quindi quando un nodo viene annotato controlla nel proprio ambiente la necessità o meno di essere un'espressione left-hand side.

L'ambiente iniziale in cui è eseguito il programma contiene le funzioni predefinite.

4.b. Errori

Il modulo di analisi semantica statica gestisce tutti gli errori a compile-time e garantisce al modulo di generazione del codice TAC un AST privo di errori.

4.b.I. Propagazione degli errori

Gli errori vengono propagati fino alla radice dell'AST. Se la radice non è priva di errori il codice non viene tradotto in TAC. Gli errori sono gestiti in due modi diversi in base al tipo di nodo:

- le espressioni che presentano errori hanno il tipo `TError`. In particolare può capitare che un'espressione abbia questo tipo in due casi:
  - c'è un errore nell'annotazione di quell'espressione. Quindi si crea un nuovo errore con un messaggio di errore opportuno;
  - c'è un errore in una o più sotto-espressioni. In questo caso si propagano gli errori (per questo il tipo `TError` ha una lista di `String`) evitando di generare nuovi errori annidati indotti (dagli errori precedenti).
- gli statement e le dichiarazioni non hanno tipo. Per questto motivo nelle loro annotazioni c'è un campo `errs :: [String]` contenente gli errori generati durante l'annotazione del nodo in questione e dei sotto-nodi.

4.b.II. Routine parziali

La definizione di routine parziali è vietata dal modulo di analisi semantica statica. In particolare è garantito che ogni flusso di esecuzione del `body` delle routine esegua indefinitivamente o incontri un `return _`.

A tal fine è presente un campo `alwaysReturn :: Bool` nelle annotazioni degli statement. Questo campo prende significato solo per statement annidati dentro funzioni, e, come suggerisce il nome:

- è vero se ritorna per ogni esecuzione dello statement. Ad esempio:

```
if true then
  return 1;
end if
```

- è falso altrimenti. Ad esempio se `a` è una variabile booleana:

```
if a then
  return 1;
end if
```

Chiaramente il controllo è conservativo: nell'esempio sopra se `a` assume sempre valore `true` per qualunque esecuzione (ma è una variabile), non è possibile saperlo durante l'analisi semantica statica.

L'annotazione di `alwaysReturn` procede dalle foglie fino alla radice dell'AST:

- le uniche foglie con `alwaysReturn = true` sono le keyword `return _` e `continue` (quest'ultima perchè è permesso ciclare all'infinito senza ritornare);
- i nodi interni dell'AST sono annotati in base a come controllano il flusso di esecuzione.

Per compilare correttamente il campo `alwaysReturn` è stato aggiunto un campo `canExit`. In generale questo campo è vero se lo statement può impedire l'esecuzione degli eventuali statement che lo seguono (allo stesso livello). Ci sono due «eccezioni»:

- il return ha `canExit = false` perchè ci interessano solo i flussi che escono senza ritornare;

- il continue ha `canExit = false` perchè, come specificato sopra, `continue` è considerato come una foglia che ritorna.

La stessa logica di controllo del flusso è stata usata per annotare un campo `unreachable` utile per evitare di generare codice inutile in TAC.

4.b.III. Dangling references

Per evitare puntatori pendenti a variabili deallocate si è deciso di vietare il return di reference nelle routine.

4.b.IV. Presenza del main

L'esecuzione del programma corrisponde all'esecuzione della routine `main`. In particolare dev'essere una procedura senza parametri in input. Se tale routine non è presente nella dichiarazione più esterna del programma, viene generato un opportuno errore.

4.c. Casting

È permesso il casting implicito. Il modulo di analisi semantica statica lo rende esplicito per la successiva traduzione in codice TAC.

Per farlo è stata implementata una funzione di supertipo `sup` che, dati due tipi, ritorna il least common ancestor in Figura 1. Il casting da τ a τ' è supportato sse $\text{sup}(\tau, \tau') = \tau'$. Tale regola vale dappertutto dove il casting è supportato (si noti che il casting non è supportato per le modalità di passaggio per riferimento e per value-result, in quanto richiedono delle espressioni di tipo left-hand side).

Il casting delle costanti avviene durante l'analisi semantica statica per risparmiare tempo a run-time.

Il casting non è esteso ai tipi composti: ad esempio non è possibile convertire direttamente un puntatore a `Integer` ad un puntatore a `Real`.

4.d. Costanti

Nelle annotazioni delle espressioni un opportuno campo `ctVal` propaga finchè determinabile a tempo di analisi semantica statica il valore costante dell'espressione. Le costanti non possono essere array o reference.

5. Generazione del Three Address Code (TAC)

La generazione del Three Address Code (TAC) è affidata ai moduli contenuti nella cartella `TAC`.

5.a. Estensioni del linguaggio TAC

Oltre alle istruzioni presentate durante le lezioni, sono state introdotte le seguenti estensioni al linguaggio TAC:

- `ArrayGen addr size`, utile per la creazione di un array di dimensione `size`;
- `Custom str`, permette di generare istruzioni personalizzate; ad esempio `END_PROGRAM` che segnala la fine del programma;
- `Comment`, consente di inserire commenti nel codice TAC.

5.b. Design e convenzioni del TAC

Alcuni aspetti rilevanti sono i seguenti:

- Le stringhe (Literal) vengono allocate in una sezione statica e sono accessibili mediante un indirizzo statico.
- Le espressioni e i parametri sono generati da sinistra verso destra.
- Gli assegnamenti multipli rappresentano, di fatto, zucchero sintattico per assegnamenti su più linee; di conseguenza, il codice TAC risultante è equivalente a quello generato da assegnamenti singoli consecutivi. Per esempio:

```
x, y, z := 0, 1, 2;
```

risulta nello stesso codice TAC di:

```
x := 0;
y := 1;
z := 2;
```

5.c. Passaggio dei parametri

La gestione delle modalità di passaggio dei parametri nelle chiamate di funzione varia a seconda della modalità utilizzata:

- **Passaggio per valore** - i parametri formali vengono copiati nei corrispondenti parametri locali, dopodiché viene eseguita la chiamata alla funzione.
- **Passaggio per reference** - ai parametri locali viene assegnata la reference dei parametri formali. All'interno del corpo della funzione, ogni utilizzo dei parametri locali viene preceduto da un'istruzione di dereferenziazione.
- **Passaggio per valore di risultato** - prima della chiamata di funzione ai parametri locali viene assegnata la reference dei parametri formali. Prima del corpo della funzione i parametri locali vengono deferenziati n copie locali (con un apice alla fine del nome). Prima di ogni istruzione `return` il valore della copia locale viene assegnato al parametro locale deferenziato.

5.d. Dettagli implementativi

Le funzioni di relazione tra stringhe e array vengono generate direttamente nel TAC, seguendo la convenzione di denominazione:

- `string<relop>'` per le relazioni tra stringhe;
- `array<relop><type>'` per le relazioni tra array, dove `<relop>` rappresenta l'operatore di relazione e `<type>` il tipo degli elementi dell'array.

Sia le espressioni booleane sia le condizioni di controllo del flusso sono valutate mediante short-circuit usando il fall-trough

Il modulo di generazione del TAC (reperibile nella cartella `TAC`) utilizza una monade di stato per generare il codice, per la costruzione del codice, consentendo di mantenere informazioni di contesto quali:

- Contatori per la creazione di temporanei, label e indirizzi statici;
- Lo stream di output per il codice generato (al contrario, come tutti gli altri stream);
- Uno stack di stream di codice per la gestione delle routine annidate;
- Uno stream per i dati statici (stringhe);
- Una lista di label che viene associata alla prossima istruzione;
- Una lista di coppie di label per la gestione di istruzioni `exit` e `continue` all'interno di loop, anche annidati;
- Un insieme che tiene traccia delle funzioni di comparazione tra stringhe e array già generate.

Nel file `TAC/State.hs`, oltre alla definizione dello stato appena descritto, sono presenti anche funzioni di supporto per la gestione dello stato.

5.e. Pretty printer

Il pretty printer del TAC utilizza la seguente notazione:

- Gli identificatori e i nomi di variabili sono annotati con la posizione nel file originale secondo la sintassi: `<ident>@<line>:<column>`;
- I commenti sono preceduti da `#`;
- Le copie locali dei parametri passati per `valres` hanno il nome del parametro seguito da un apice `param'`;
- Anche le funzioni di relazione tra stringhe e array presentano un apice alla fine del nome `rel'`;
- Le variabili che puntano a dati statici sono precedute da `$`.

Bibliografia

[1] M. Comini, «Front-end di un compilatore per un linguaggio imperativo: note generali», 2024.